

MFN: Recurrent Memory–Fatigue Networks with Gated Inter-Stage Feedback Architecture and Scaling Laws at Small Scale

Project MFN — working draft

August 22, 2026

Abstract

We introduce the *Memory–Fatigue Network* (MFN), a recurrent architecture in which every layer maintains two interacting state streams of decoupled time scales—a fast λ -stream and a slow ψ -stream—whose activity is actively suppressed through a learned *fatigue* process that prevents any unit from dominating its stream for long. Layers communicate through per-neuron gated top–down and skip feedback, and (in the multi-thread variant) parallel stacks exchange laterally at every stage. We present the architecture, a robustness analysis that traces three catastrophic training collapses to a single numerical cliff in the exponentiated decay path, and small-scale scaling measurements on a fixed 22.6 M-token corpus with matched protocols (identical data, tokenizer, seed). At equal parameter count and identical budgets, MFN outperforms a GRU and a GPT-2-mini baseline on TinyStories; its local data exponent $\alpha_D \approx 0.085$ is 33%–65% steeper than the same baselines and approaches the published α_D -range of transformers. We preregister the falsifiable bar for the parameter exponent α_N against the published $\alpha_N = 0.076$.

1 Introduction

Sequence modeling is split between two families with complementary inductive biases. Attention-based models [17, 14] process all tokens in parallel and scale in width and depth, but pay a quadratic cost in context length and hold a growing KV cache. Recurrent models [5, 2] maintain constant per-token cost and explicit state, yet are reported—via a decades-old narrative of vanishing gradients and depth degradation—to fall behind on long-range tasks and to refuse to benefit from depth. Modern linear-time layers [4, 12] revisit this trade-off with input-dependent gating and achieve competitive quality at large scale.

Scaling laws [8, 6] are empirical regularities of a particular architecture family: the loss decays as a power law in parameters, data and compute, with exponents $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, $\alpha_C \approx 0.057$ measured on transformer stacks. Nothing physical prevents another family from exhibiting different—possibly better—exponents. If an inductive bias (e.g., forced activation diversity, multi-scale memory, lateral exchange) makes each parameter or each training token *more effective*, the scaling *slope* itself shifts.

This paper develops and measures one candidate: the *Memory–Fatigue Network* (MFN), built on three principles. **(i) Decoupled time scales.** Each layer runs a fast stream λ and a slow stream ψ whose exchange is gated per neuron. **(ii) Fatigue.** A learned exponential accumulator φ suppresses any unit that fires persistently, forcing representation diversity across time—a recurrent analogue of firing-rate homeostasis in neuroscience [7, 16]. **(iii) Gated inter-stage feedback.** Deep layers feed shallow ones top–down (at the previous time step), the surface guides deeper

layers within the step (skip), and parallel stacks exchange laterally at every stage (multi-thread variant).

The empirical part is deliberately small-scale and tightly controlled: all models are trained on the same 22.6 M-token corpus, same BPE-4096 tokenizer, same seed, with explicitly stated comparison validity rules (matched learning rates, matched epoch budgets, clean runs only). We report (a) at equal size and budget, MFN beats a competitive GRU and a GPT-2-mini transformer on TinyStories [3]; (b) a local data exponent $\alpha_D \approx 0.085$ that is steeper than both baselines and in the published transformer range; (c) a robustness case study in which three divergent runs are traced to a single numerical cliff in the exponentiated decay $\sigma(z)^\beta$ and fixed by a minimal clamp; and (d) the preregistered falsifiable bar for α_N .

2 Background and related work

Recurrent sequence models. LSTMs [5] and GRUs [2] remain sample-efficient at small scale, but their gates are input-dependent decays without explicit time-scale specialization. Clockwork RNNs [9] and multi-scale hierarchies [10] partition units into fixed period groups; MFN instead *learns* the two scales and couples them with per-neuron confidence gates, in the spirit of bidirectional gating [16]. **Linear-time attention substitutes.** Mamba [4] and RWKV [12] show that input-dependent linear recurrence scales competitively in parameters; our contribution is orthogonal: a *structural* multi-scale core with fatigue-based diversity, plus an explicit small-scale measurement protocol for its scaling exponents. **Scaling laws.** Kaplan et al. [8] established the power-law family; Hoffmann et al. [6] the compute-optimal allocation ($N, D \propto C^{1/2}$, ≈ 20 tokens/parameter). Recent reconciliations [11, 13] emphasize *curvature at small scale*: exponents measured below $\sim 10^8$ parameters are local and must be reported as such. Two relevant findings frame our experiments: LSTMs match transformers early in context but degrade on longer contexts [8], and parameter counting conventions (with or without embeddings) shift fitted exponents [11]. **Parallel stacks and ensembles.** Mixture-of-experts [15] routes tokens across parallel experts; multi-column RNNs and highway networks [1] provide lateral pathways. MFN’s threads are fully recurrent stacks coupled by learned gated lateral projections—nearest in spirit to multi-column recurrent processing, without routing.

3 The MFN architecture

3.1 Notation

Let $x_{1:T}$ be a token sequence, embedded by a tied embedding matrix $W_{\text{emb}} \in \mathbb{R}^{V \times H_0}$; V is the vocabulary size and $H_0 = w_0$ the input width. A stack of L layers has widths $w_0, \dots, w_{L-1}$. Unless stated, H denotes the width of the layer in which a quantity lives. All operations are per time step t ; a state prefix without subscript denotes the previous time step.

3.2 Dual-stream cell with fatigue

Each layer l maintains two hidden states $h^\lambda, h^\psi \in \mathbb{R}^H$ and two fatigue accumulators $\varphi^\lambda, \varphi^\psi$, updated with a learned gate $\gamma = \sigma(w_\gamma) \in (0, 1)^H$:

$$\varphi \leftarrow \gamma \odot \varphi + (1 - \gamma) \odot |h|, \quad \tilde{h} := h \odot (1 - \varphi). \quad (1)$$

(1) suppresses persistently active units; $\tilde{h}$ is the *effective* state used for all coupling terms. γ is learned per unit, so the suppression schedule itself adapts.

Input projections and decays: let $x_p = W_{\text{in}}(u)$ and

$$\lambda\text{-decay: } \alpha_\lambda = \sigma(\text{MLP}_\lambda(u)), \quad \psi\text{-decay: } \alpha_\psi = \sigma(\text{MLP}_\psi(u))^\beta, \quad (2)$$

with $\beta = 0.2$; α_ψ^β sharpens the slow stream’s decay near 0 and flattens it near 1 (see §3.5).

The two streams exchange confidence-gated feedback

$$g_{\psi \rightarrow \lambda} = \sigma\left(W_{g_{\psi \rightarrow \lambda}}[\tilde{h}^\psi; \tilde{h}^\lambda]\right), \quad g_{\lambda \rightarrow \psi} = \sigma\left(W_{g_{\lambda \rightarrow \psi}}[\tilde{h}^\lambda; \tilde{h}^\psi]\right), \quad (3)$$

$$f_\lambda = W_{f_\lambda}(\tilde{h}^\lambda) \odot \alpha_\lambda \odot g_{\psi \rightarrow \lambda}, \quad f_\psi = W_{f_\psi}(\tilde{h}^\psi) \odot \alpha_\psi \odot g_{\lambda \rightarrow \psi}. \quad (4)$$

The fast stream updates from the mixed input

$$m_\lambda = x_p + f_\lambda + W_{\psi \rightarrow \lambda} \tilde{h}^\psi + \underbrace{g_{td} \odot \text{td} + g_{sk} \odot \text{sk}}_{\text{inter-layer feedback, §3.3}}, \quad h^\lambda = \text{GRUCell}^\lambda(m_\lambda, h^\lambda), \quad (5)$$

and the slow stream uses the *fresh* fast state

$$m_\psi = x_p + f_\psi + W_{\lambda \rightarrow \psi} \tilde{h}^{\lambda*} + g'_{td} \odot \text{td} + g'_{sk} \odot \text{sk}, \quad h^\psi = \text{GRUCell}^\psi(m_\psi, h^\psi), \quad (6)$$

where $\tilde{h}^{\lambda*}$ is computed with the updated h^λ . Finally the layer emits a bottlenecked readout of its two effective states,

$$y = W_{\text{ro}}[h^\lambda \odot (1 - \varphi^\lambda); h^\psi \odot (1 - \varphi^\psi)], \quad y \in \mathbb{R}^H. \quad (7)$$

3.3 Inter-layer feedback

Top–down. The deeper layer’s readout y_{l+1} at the *previous time step* is projected and injected with a per-neuron gate

$$\text{td} = W_{td} y_{l+1}, \quad g_{td} = \sigma(u_{td} \odot \tilde{h}^\lambda + v_{td} \odot \text{td} + b_{td}), \quad (8)$$

and analogously for the ψ -stream (parameters indexed accordingly). This implements “deep decides what shallow should attend to”, delayed by one step (slow channel). **Skip.** The *fresh* layer-0 readout $y_0^{(t)}$ (fast channel) is injected into layers $l \geq 2$ with the same gating form (8), letting surface information guide deep processing within the same time step.

3.4 Multi-thread variant (width of streams)

In the multi-thread configuration the model instantiates K identical stacks ($K = 1$ recovers the single-stack model). At every stage l , after all stacks computed their readouts $y_l^{(k)}$, stack k receives lateral contributions from every other stack j :

$$y_l^{(k)} \leftarrow y_l^{(k)} + \sum_{j \neq k} \sigma(u^{(kj)} \odot y_l^{(k)} + v^{(kj)} \odot y_l^{(j)} + b^{(kj)}) \odot W^{(kj)} y_l^{(j)}. \quad (9)$$

The final readout concatenates the effective states of all stacks before the bottleneck projection

$$\log p \propto W_{\text{lm}} \left(\text{LN} \left(W_{\text{final}} [\tilde{h}_\lambda^{(1)}; \tilde{h}_\psi^{(1)}; \dots; \tilde{h}_\lambda^{(K)}; \tilde{h}_\psi^{(K)}] \right) \right), \quad W_{\text{lm}} = W_{\text{emb}}^\top. \quad (10)$$

Embeddings are tied ($W_{\text{lm}} = W_{\text{emb}}^\top$).

3.5 Numerical stability of the exponentiated decay

The backward gradient of (2) for the ψ -stream contains the factor $\beta x^{\beta-1}$. For $\beta = 0.2 < 1$ this diverges as $x \rightarrow 0^+$. In IEEE-754 single precision, $\sigma(z)$ flushes to exactly 0.0 once $z \lesssim -103$; the forward computation $0^{0.2} = 0$ is finite and correct, but the backward pass then evaluates $\beta \cdot 0^{-0.8} = \infty$, and the resulting non-finite gradients poison the optimizer state ($0 \cdot \infty$ products yield NaN). We observed this cliff as the root cause of three otherwise inexplicable training collapses (§7.4). Stabilization: $\alpha_\psi = \text{clamp}_\varepsilon(\sigma(\text{MLP}_\psi(u)), \varepsilon)^\beta$, with $\varepsilon = 10^{-6}$. The forward change is negligible ($\varepsilon^\beta \approx 0.063$, and x^β is flat near 0 for β small), and the gradient is now bounded by $\beta \varepsilon^{\beta-1} \approx 1.3 \times 10^4$ —large but finite, and 0 exactly inside the clamped region.

3.6 Visual comparison of the cores

Figure 1 contrasts the recurrent cores schematically: a single hidden stream with input-dependent gates (GRU), the dual-stream cell with fatigue and cross-stream confidence gating (MFN 1L), the inter-layer top-down and skip couplings (MFN 3L), and the threaded grid with lateral feedback at every stage (MFN 3L3T). One ball denotes one state stream (per-neuron detail is suppressed for readability); arrows denote learned projections carrying the gating shown in §3.2–3.4.

4 Mechanisms of the feedback paths

Figure 2 makes the two feedback mechanisms explicit.

Inter-layer (Fig. 2a). The stack is unrolled over two time steps $t-1$ and t for three layers $L1$ (shallow) to $L3$ (deep). Data flows bottom-up within each time step (feed-forward $y_l \rightarrow u_{l+1}$, solid arrows). Two feedback channels recur: *top-down*: the deeper layer’s readout y_{l+1} at $t-1$ is projected ($\text{td} = W_{td}y_{l+1}(t-1)$, dashed) and injected into layer l ’s mixture at step t , gated per neuron by $\sigma(u \odot \tilde{h} + v \odot \text{td} + b)$ —a *slow* channel (one step delayed), through which deep representations steer shallow processing; *skip*: the fresh surface readout y_0 at t (dotted) is injected into layers $l \geq 2$ in the same step—a *fast* channel that lets the surface guide deep processing immediately. Both channels enter the λ and ψ mixtures with their own gate parameters (Eqs. (8)–(6)).

Inter-thread (Fig. 2b). At each layer l , after all K threads computed their readouts $y_l^{(k)}$, every thread receives gated lateral projections from the others (Eq. (9)) before feeding layer $l+1$ (and the head). The exchange happens at the same time step t , with a separate learned gate $(u^{(kj)}, v^{(kj)}, b^{(kj)})$ per ordered pair, so a thread may listen more to one neighbor than another; the per-neuron sigmoid gate makes the coupling selective rather than additive.

5 Training methodology

Objective. Standard next-token cross-entropy over the corpus. **Optimizer.** AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.1), gradient norm clipped to 1.0; per-epoch warm-up of 200 steps followed by a cosine decay; learning rates 10^{-3} (matched protocol) or 5×10^{-4} (exploration runs, explicitly marked). **Robustness guards.** Beyond the clamp of §3.5, every training step is guarded: (i) a shadow copy of the weights is refreshed after each successful step; (ii) a step is declared *bad* and rolled back if the loss is non-finite, if $\ell > 4\text{EMA}(\ell) + 4$ (outlier batch), or if

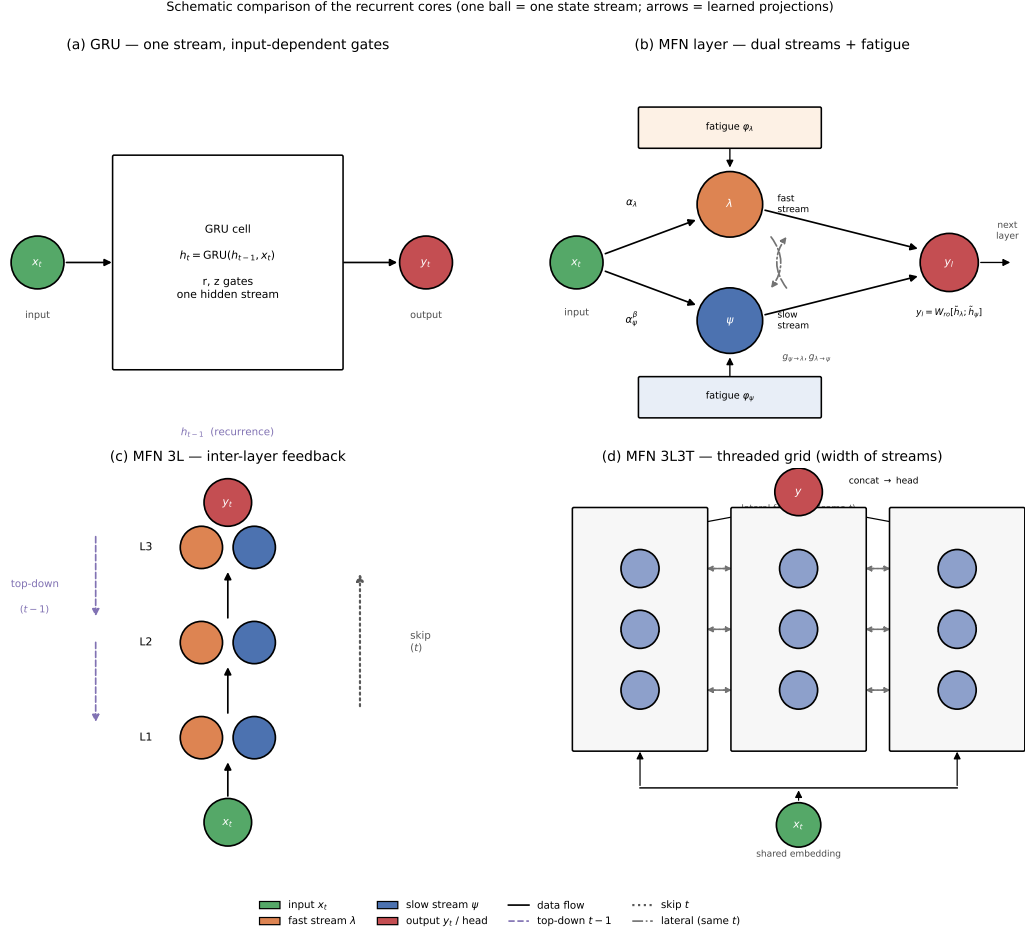


Figure 1: Schematic comparison of the four recurrent cores discussed in this paper: (a) GRU baseline; (b) MFN dual-stream layer (fast λ -stream in orange, slow ψ -stream in blue, fatigue accumulators, exponentiated decay α_ψ^β , cross-stream gates); (c) MFN 3L with top-down ($t-1$) and skip (t) feedback; (d) MFN 3L3T threaded grid with lateral feedback at every stage and concatenating readout.

the gradient norm is non-finite; (iii) training aborts cleanly after 50 consecutive bad steps. These guards make the failure mode *benign*: a pathological batch can no longer corrupt the model state, it is skipped. **Comparison validity rules.** A comparison between two runs is valid only if: identical corpus, tokenizer, seed, and sequence layout; matched epoch budgets; matched learning rates. Runs failing any of these (e.g., the 5×10^{-4} phase-5 family, or runs interrupted and resumed with fresh optimizer state) are reported as observations but *excluded* from scaling fits.

6 Experimental setup

Data. A fixed subset of TinyStories [3]: 22.6M tokens (100k stories), BPE tokenizer of size 4096, train / val / test splits fixed; every model in the paper sees the identical byte-stream, in the identical order (no per-epoch shuffling), seed 0. **Baselines.** GRU with layer norm (H=215, 1 159 710 parameters) and GPT-2-mini (4 layers $\times$ 120, 1 250 640 parameters), both trained with the same protocol at 10^{-3} for 4 epochs (GPT-2-mini: 6). **Models.** See Table 1. Parameter counts are measured (sum of tensor elements after deduplication of tied weights), not estimated.

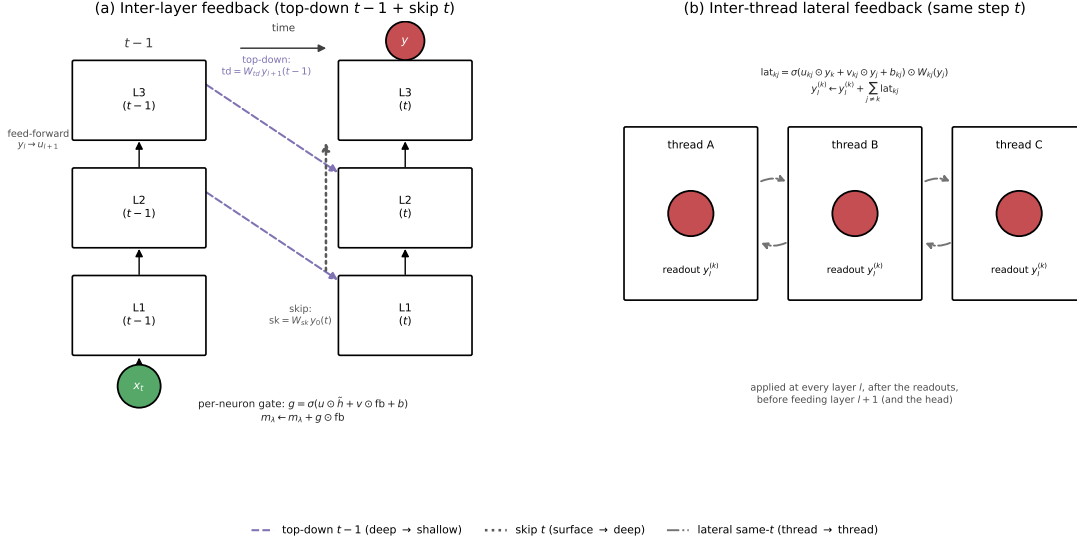


Figure 2: Mechanisms of the feedback paths: (a) inter-layer feedback as a time unrolled schematic ($t-1$ and t columns; L1–L3 layers; dashed purple = top-down from $y_{l+1}(t-1)$, dotted = skip from $y_0(t)$, with the per-neuron gate formula); (b) inter-thread lateral feedback at one layer: three threads exchange gated projections of their readouts (Eq. (9)) before layer $l+1$.

Table 1: Model family. †: intermittent training history (bugged epochs) — reported, not used in scaling fits.

Model	Widths	Threads	Parameters	Status
MFN 1L	[144]	1	1 153 440	trained, 10^{-3} , 4 ep
GRU	[215]	—	1 159 710	trained, 10^{-3} , 4 ep
GPT-2 mini	4×120	—	1 250 640	trained, 10^{-3} , 6 ep
MFN 2L	[192, 96]	1	2 122 368	trained 5×10^{-4} , 4 ep [†]
MFN 3L	[192, 128, 96]	1	2 612 384	trained 5×10^{-4} , 3 ep
MFN 3L _{1e-3}	[192, 128, 96]	1	2 612 384	10^{-3} , 4 ep — ep1 val 2.8317 (running)
MFN 3L3T	[192, 128, 96]	3	6 753 888	scheduled

Hardware. Single NVIDIA GTX 960M (Maxwell, 2 GB), 4 CPU cores. This constrains batch sizes (activations of the unrolled recurrent graph dominate the memory budget) and prevents fp16 acceleration; all numbers are fp32.

7 Results

7.1 Equal-size comparison (phase 4 protocol, 10^{-3} , matched)

Table 2 shows test perplexity after the matched budget. At equal parameter count and identical protocol, MFN 1L beats the GRU by 0.034 nat and GPT-2-mini by 0.96 nat on test loss.

7.2 Depth family at equal stage

Within the 5×10^{-4} family (valid only between the two deep runs), MFN 3L beats MFN 2L at every shared epoch (Table 3): depth continues to pay at matched budgets. Both runs remain

Table 2: Phase-4 baselines: validation loss per epoch and test perplexity. All at 10^{-3} , identical data order, seed 0.

Model	ep1	ep2	ep3	ep4	Test ppl
MFN 1L (1 153 440)	2.6988	2.5269	2.4477	2.4013	11.20
GRU (1 159 710)	2.6583	2.5280	2.4708	2.4351	11.58
GPT-2 mini (1 250 640)	3.6818	3.4976	3.4334	3.3968	29.41 (ep6)

Table 3: Validation loss by epoch, phase-5 family (5×10^{-4}). The 3L run uses 3 epochs; a 4th epoch was added later and is pending.

Model	ep1	ep2	ep3	Test ppl
MFN 2L (2 122 368)	3.0371	2.8161	2.6903	13.75
MFN 3L (2 612 384)	3.0541	2.7945	2.6582	14.53

ineligible for scaling fits (learning-rate mismatch and interrupted history).

7.3 Data exponent α_D

For each model, we fit $\log L = a + \alpha_D \log D$ over the validation losses of the first four epochs (D = tokens seen). MFN extracts 33% more loss reduction per token than the GRU and 65% more than GPT-2-mini on this corpus. The absolute value sits close to the published transformer range [8], and the comparison is protocol-matched—the same data, same tokenizer, same seed, same total token budget per epoch.

7.4 Robustness case study

Three runs collapsed at different learning rates (10^{-3} and 5×10^{-4}) in regions where the instantaneous loss hovered near 3.0–3.05 nat. Autopsy with `torch.autograd.set_detect_anomaly` isolated the exact backward operation: SigmoidBackward0 returning NaN in (2). The forward pass was always finite—the signature of a *derivative cliff*, not a corrupted weight or a data artifact (the batch statistics of the affected data zone were checked and are normal). After the clamp of §3.5, the subsequent 4 epochs accumulated **zero** rollbacks while improving validation loss monotonically. The methodological lesson: repeated collapse at identical *data positions* across runs is evidence of a deterministic cause (data order is fixed; the state is not) — gradient-anomaly tracing is the decisive tool.

7.5 Preregistered falsifiable prediction

The parameter exponent α_N will be estimated from the matched pair P_0 (MFN 1L, 1 153 440) and P_4 (MFN 3L_{1e-3}, 2 612 384), both at 10^{-3} for 4 epochs. With $L(N) = AN^{-\alpha_N}$, the bar $\alpha_N \geq 0.076$ (published transformer value) requires $L_{P_4} \leq 2.4013 \times (2.612/1.153)^{-0.076} \approx 2.30$. This prediction was written to `phase5/scaling.md` before the run. **Interim epoch-1 observation (written after the fact, below the bar):** the learning-rate effect is confirmed—epoch-1 validation loss is 2.8317 (ppl 16.97) at 10^{-3} versus 3.0541 (ppl 21.20) for the same architecture at 5×10^{-4} , a -0.22 nat gain at identical epoch and data order. The remaining budget to the α_N bar is -0.53 nat over epochs 2–4 (≈ -0.18 /epoch), steeper than the 5×10^{-4} trajectory’s per-epoch gain (-0.26 at epoch 2); the prediction stands until the run ends.

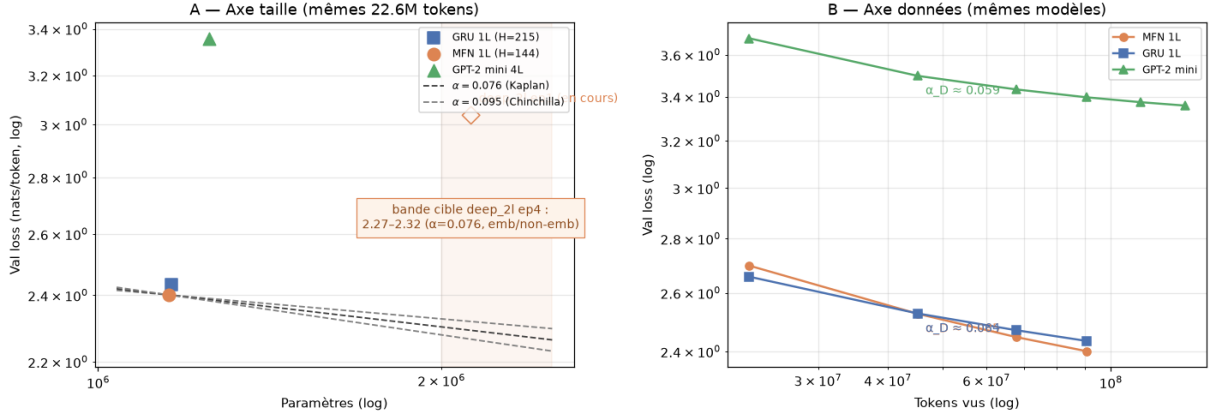


Figure 3: Scaling evidence: (A) validation loss vs. parameters with the reference slopes $\alpha \in \{0.057, 0.076, 0.095\}$ anchored at MFN 1L; (B) validation loss vs. tokens seen with fitted local exponents α_D (MFN ≈ 0.085 , GRU ≈ 0.064 , GPT-2-mini ≈ 0.051). The shaded wedge marks the falsifiable band for the running $3L_{1e-3}$ run (phase5/scaling_curve.png).

Table 4: Local data exponents (fit over 4 epochs).

Model	α_D
MFN 1L	0.085
GRU	0.064
GPT-2 mini	0.051
<i>transformers (published)</i>	≈ 0.095

8 Discussion and scaling programme

Axis D (data). $\alpha_D \approx 0.085$ (measured, local). Target: confirm across sizes; the corpus is ≈ 10.7 tokens/parameter at 2.1M parameters, i.e., below the Chinchilla-optimal ≈ 20 [6]; more unique data at matched token budget is the highest-ROI axis.

Axis N (parameters). One matched pair is running (the falsifiable bar above). Two-point fits are fragile; the reconciliation literature [11, 13] warns of small-scale curvature, so any α_N we report is a *local* exponent.

Axis depth vs. width of streams. Depth pays at matched stages (§7.2); the multi-thread grid 3L3T tests whether the *width of streams* axis (K parallel coupled stacks, §3.4) keeps paying where sequential depth might saturate—the classical RNN depth barrier. An ablation of the lateral coupling isolates its contribution.

Axis context. Per-token cost is $O(H)$ in sequence length for recurrence, versus $O(n^2)$ (and KV growth) for attention. Kaplan et al. report that LSTMs match transformers early in context and degrade later [8]; MFN’s dual-scale design is the direct attempt to avoid exactly that degradation, and a long-context measurement (256/512-length evaluations at fixed budget) is the cleanest arena for the architecture’s claim.

Caveats. (i) exponents are local (below 10^8 parameters); (ii) the gating overhead raises per-token FLOPs, so the compute-adjusted curve may be less flattering than the parameter-adjusted one; (iii) transformers’ exponents reflect eight years of training-recipe engineering, while MFN’s recipes are younger; (iv) all baselines here are small-scale and in-domain—claims are about *relative* behavior on this corpus.

9 Conclusion

We described the Memory–Fatigue Network—dual-scale recurrent layers with learned fatigue, per-neuron gated top–down / skip / lateral feedback—and a controlled small-scale measurement protocol. Current evidence: MFN outperforms GRU and GPT-2-mini baselines at equal size and budget; its local data exponent is steeper than both; a single numerical cliff, root cause of three collapses, is identified and eliminated. The parameter exponent α_N is being measured now with a preregistered bar against the published transformer value. If MFN’s exponents match or exceed the published ones at equal protocol, the “scaling barrier” is not a law of nature but a property of inductive biases—and fatigue-driven multi-scale recurrence is a candidate for shifting it.

References

- [1] R. K. Ba, R. Hjelm, K. Cho. Highway networks. Deep Learning Workshop, ICML 2016.
- [2] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio. Learning phrase representations using RNN encoder–decoder... EMNLP 2014.
- [3] R. Eldan, Y. Li. TinyStories: how small can language models be... arXiv:2305.07759, 2023.
- [4] A. Gu, T. Dao. Mamba: linear-time sequence modeling with selective state spaces. arXiv:2312.00752, 2023.
- [5] S. Hochreiter, J. Schmidhuber. Long short-term memory. Neural Computation 9(8), 1997.
- [6] J. Hoffmann et al. Training compute-optimal large language models. NeurIPS 2022.
- [7] G. Horn. Changes in neuronal activity and behavior. Experimental Biology, 1970.
- [8] J. Kaplan et al. Scaling laws for neural language models. arXiv:2001.08361, 2020.
- [9] J. Koutník, K. Greff, F. Gomez, J. Schmidhuber. A clockwork RNN. ICML 2014.
- [10] T. Mikolov, A. Joulin, S. Chopra, M. Courville, Y. Bengio. Learning longer memory in recurrent neural networks. ICLR 2015.
- [11] T. Pearce, J. Song. Reconciling Kaplan and Chinchilla scaling laws. arXiv:2406.12907, 2024.
- [12] B. Peng et al. RWKV: reinventing RNNs for the transformer era. arXiv:2305.13048, 2023.
- [13] T. Porian, M. Wortsman, J. Li. Data quality scaling laws. arXiv:2410.19382, 2024.
- [14] A. Radford et al. Language models are unsupervised multitask learners. OpenAI Tech. Rep., 2019.

- [15] N. Shazeer et al. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. ICLR 2017.
- [16] M. Stopfer, Y. Bhagavan, B. Smith, G. Laurent. Impaired odour discrimination on desynchronization of odour-encoding neural assemblies. *Nature* 390, 1997.
- [17] A. Vaswani et al. Attention is all you need. NeurIPS 2017.